Mnemosyne — Causal Memory Engine: Build Spec

> Handoff for Claude Code. This document specifies a causally-indexed, event-sourced memory substrate for LLM agents. The novel part is the **causal graph**: not just storing facts, but storing timestamped events and the directed cause→effect edges between them, then retrieving by walking those edges.
> Baseline (associative/vector retrieval) is included only as a comparison harness.
> Spend the effort on the causal engine.

---

0. What we're building (one paragraph)

A memory service that ingests conversational/log text, extracts **timestamped events** (not static facts), appends them to an immutable event log, and infers **directed causal edges** between events. At query time it anchors on the event(s) matching the query, then **traverses the causal graph** backward (root cause) and forward (consequences) to return a time-ordered, noise-free chain — instead of a bag of semantically-similar chunks.

---

1. Tech stack

- **Language:** Python 3.11+
- **API:** FastAPI + uvicorn
- **Graph store:** Neo4j (via `neo4j` driver) OR start with NetworkX in-memory for v0, behind an interface so it's swappable
- **Vector store:** Qdrant (we're already in that ecosystem) for the semantic-anchor step; `qdrant-client`
- **Embeddings:** `sentence-transformers` (e.g. `all-MiniLM-L6-v2`) for v0; pluggable
- **LLM (for event/causal extraction):** Anthropic API, `claude-haiku` for extraction, `claude-sonnet` for causal judgment. Read key from env, never hardcode.
- **Validation:** Pydantic v2
- **Tests:** pytest

Keep the graph backend and embedding model behind interfaces (Protocol classes) so they can be swapped. Do NOT couple business logic to Neo4j or Qdrant specifics.

---

2. Core data model

Event (the only thing we store as ground truth)

```python
class Event(BaseModel):
    id: str                          # uuid
    summary: str                     # one-line natural language
    detail: str                      # full text
    # --- bi-temporal: this is critical, do not collapse to one timestamp ---
    occurred_at: datetime            # when the event happened in the world
    learned_at: datetime             # when the system ingested/learned it
    # --- semantic ---
    embedding: list[float]           # for the anchor step
    tags: list[str]                  # extracted entities/topics
    # --- provenance ---
    source_id: str                   # which document/session it came from
```

CausalEdge (the novel index)

```python
class CausalEdge(BaseModel):
    id: str
    cause_id: str                    # Event.id
    effect_id: str                   # Event.id
    relation: str                    # "caused" | "triggered" | "led_to" | "enabled" |
    confidence: float                # 0..1, REQUIRED — never store an edge without it
    evidence: str                    # why we think this is causal (the cue/justificati
    method: str                      # "temporal+linguistic" | "llm_judgment" | "manual
    occurred_delta_s: int            # effect.occurred_at - cause.occurred_at, in secon
```

Invariants to enforce:

- An edge is only valid if `cause.occurred_at <= effect.occurred_at` (causes precede effects). Reject/flag violations.
- Confidence is mandatory. Edges below a configurable threshold (default 0.55) are stored but excluded from default traversal.
- The event log is **append-only**. Never mutate an event. Corrections are new events that supersede (add a `supersedes` edge type if needed).

---

3. The pipeline (build in this order)

Stage A — Event extraction

Input: raw text (a chat turn, a log block, a doc). Output: zero or more `Event`s.

- Prompt the LLM to extract discrete events with their `occurred_at` (resolve relative times like "this afternoon", "6 minutes later" against the message timestamp).
- Each event gets `summary`, `detail`, `tags`.
- Embed `summary` + `detail`, write to Qdrant; write the Event node to the graph store.

Stage B — Causal edge inference (THE HARD PART — focus here)

For each new event, consider candidate cause events and decide which causal edges to create.

Use a **three-signal scoring** approach. Do not rely on the LLM alone, and do not rely on heuristics alone — combine them:

1. **Temporal precedence + proximity.** A candidate cause must occur before the effect. Score decays with the time gap (configurable window, e.g. events within 30 min are strong candidates; beyond that, weak). This prunes the candidate set cheaply.
2. **Linguistic / co-occurrence cues.** Look for explicit causal markers in the source text linking the two ("because", "due to", "which caused", "as a result", "led to", "triggered"). Also entity overlap — shared subjects/objects raise the prior.
3. **LLM causal judgment.** For surviving candidate pairs, ask the LLM (Sonnet): "Did event A causally contribute to event B? Answer with a relation type and a confidence 0–1 and a one-sentence justification." Force structured JSON output.

Final `confidence` = a weighted blend of the three signals (start simple: e.g. `0.3*temporal + 0.2*linguistic + 0.5*llm`, then tune). Store `evidence` and `method`.

> Be honest in code comments that this is **approximate** causality (temporal precedence
> - association + LLM judgment), not true causal inference. That's the accepted practical tradeoff — don't oversell it.

Stage C — Consolidation (cross-session reinforcement)

When a new event closely matches an existing one (high embedding similarity + tag overlap

- compatible time), don't duplicate. Instead increment a `reinforcement_count` and bump confidence on its edges. Repeated independent confirmation = higher-trust memory. Also apply **temporal decay**: a `salience` score on each event that decays with age unless reinforced, used to break ties at retrieval.

4. Retrieval (the payoff)

```
POST /retrieve  { "query": str, "k": int, "mode": "causal" | "associative" }
```

Associative mode (baseline): embed query → Qdrant top-k → return chunks. No ordering. This exists ONLY so we can demonstrate the difference. Keep it minimal.

Causal mode (the real thing):

1. **Anchor:** embed the query, find the best-matching event(s) — the "effect" the user is asking about.
2. **Traverse backward:** follow `cause_id` edges from the anchor to root cause(s), only crossing edges above the confidence threshold. Collect the chain.
3. **Traverse forward (optional, configurable):** follow `effect_id` edges to capture resolution/consequences.
4. **Order by `occurred_at`** and return the chain with edge relations + confidences.
5. **Synthesize:** hand the ordered chain to the LLM to produce the "why" narrative, citing event IDs and timestamps.

Return shape:

json

```
{
  "anchor_event_id": "...",
  "chain": [ {"event": {...}, "incoming_relation": "caused", "confidence": 0.82}, ..
  "answer": "natural-language causal explanation",
  "excluded_distractors": ["id1","id2"]   // semantically-similar events NOT on any
}
```

The `excluded_distractors` field is important — it's the demonstrable proof that causal traversal ignores keyword-similar noise that associative retrieval would wrongly surface.

---

5. API surface (v0)

Method	Route	Purpose
POST	`/ingest`	text in → events extracted, edges inferred, stored

Method	Route	Purpose
POST	`/retrieve`	query in → causal chain or associative chunks out
GET	`/graph`	dump nodes + edges (for visualization / debugging)
GET	`/event/{id}`	inspect a single event + its edges
GET	`/health`	liveness

---

6. Acceptance test (build this scenario as a fixture)

Seed the incident scenario from the demo so behavior is verifiable:

- e1 `14:02` Config change: cache TTL 3600→30s
- e2 `14:08` Cache hit-rate collapses
- e3 `14:14` DB connection pool saturates
- e4 `14:19` Checkout 500s spike ← query target
- e5 `14:41` Rollback of the config change
- e6 `14:55` Recovery confirmed
- d1 `3 weeks ago` Cache-warming cron (distractor, shares "cache" tags)
- d2 `4 months ago` Checkout redesign doc (distractor, shares "checkout" tags)

Test 1 (causal): `retrieve("Why did checkout start throwing 500s?", mode="causal")` must return the chain e1→e2→e3→e4 (+ optionally e5→e6), ordered by time, with e1 as root, and d1/d2 in `excluded_distractors`.

Test 2 (associative): same query, `mode="associative"` should surface d1 and/or d2 among top-k (demonstrating the failure mode).

Test 3 (invariant): attempting to create an edge where cause occurs after effect is rejected.

Test 4 (reinforcement): ingesting a near-duplicate of e1 increments reinforcement_count rather than creating a second node.

---

7. Build order for Claude Code

1. Scaffold: FastAPI app, Pydantic models, graph-store interface + NetworkX impl, config via env.

2. Event extraction (Stage A) + `/ingest` with the LLM extraction prompt.
3. **Causal edge inference (Stage B) —** the three-signal scorer. Unit-test each signal in isolation.
4. Causal retrieval + traversal (Section 4) + `/retrieve`.
5. Associative baseline (minimal) for comparison.
6. The acceptance-test fixture (Section 6) and tests.
7. Consolidation + decay (Stage C) last — it's an enhancement, not a blocker.
8. Swap NetworkX → Neo4j and in-memory → Qdrant behind the interfaces once v0 passes tests.

---

8. Things to get right / common traps

- **Bi-temporal from day one.** Retrofitting `learned_at` later is painful. Store both timestamps now.
- **Never store a causal edge without confidence + evidence.** Unexplained edges are useless and erode trust.
- **Causes must precede effects.** Enforce as a hard invariant, not a soft check.
- **Keep extraction and judgment as separate LLM calls** with structured (JSON) outputs; validate with Pydantic, fail loudly on malformed output.
- **Don't oversell causality.** Comment clearly that this is temporal+associative+LLM approximation.
- **Append-only log.** All "edits" are new events. This is what makes time-travel queries ("what did we believe at T?") possible later.
- **Interfaces over implementations** for graph + vector + embeddings so the stack is swappable.

---

9. Reference points worth reading

- **Graphiti** (the engine under Zep) — closest mature implementation of bi-temporal causal/episodic memory. Study its temporal model.
- **GraphRAG** (Microsoft) — for the entity/relationship extraction patterns.
- Contrast with **Mem0** / **MemGPT** / **LangChain Memory** to articulate the gaps we're closing (no spaced repetition, no temporal decay, no cross-session reinforcement, no causal index).